Machine Learning in Python

Lecture 3: Tensor operations and gradient descent

Przemysław Rola

Department of Mathematics, Krakow University of Economics

Plan

1. Tensor operations

2. How a network learns

Based on F. Chollet, M. Watson, *Deep Learning with Python*, 3rd ed., Sec. 2.3–2.4.

Tensor operations

What a dense layer computes

```
layers.Dense(512, activation="relu")
```

computes, for a batch of inputs `x`,

```
output = relu(matmul(x, W) + b)
```

- `matmul(x, W)`: matrix product (all weighted sums at once),
- `+ b`: add the bias of every unit,
- `relu`: element-wise $\max(\cdot, 0)$.

Three operations on tensors. A whole network is just a long chain of such operations.

Element-wise operations

Applied to every entry independently: `relu`, `+`, `-`, `*`, ...

```
def naive_relu(x):              # x: matrix
    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] = max(x[i, j], 0)
    return x
```

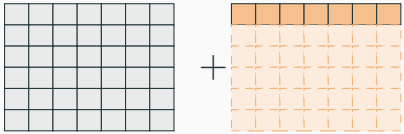
```
import numpy as np
z = x + y                       # element-wise sum
z = np.maximum(z, 0.0)          # element-wise relu
```

- Same result; the NumPy version is about 100 times faster (book: 0.02 s vs 2.45 s).
- NumPy calls optimised low-level code; the loop runs in slow Python.
- Rule: **avoid Python loops over entries**.

Broadcasting: adding a vector to every row

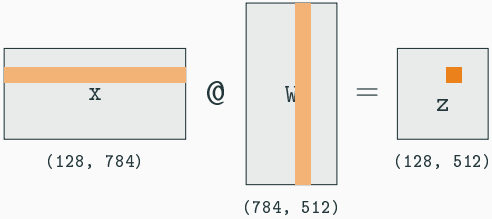
```
X = np.random.random((32, 10))   # batch
b = np.random.random((10,))      # biases
Z = X + b                        # shape (32, 10)
```

- Shapes differ: `(32, 10)` and `(10,)`.
- NumPy behaves as if `b` were copied into all 32 rows (no copy is actually made).
- That is exactly what we need for `x @ W + b`.



The dot product and the matrix product

- **Dot product** of two vectors of the same length: $x \cdot y = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n$ — a weighted sum, exactly as in the perceptron.
- **Matrix product** `x @ W`: entry (i, j) = dot product of row i of `x` with column j of `W`.



rule: `(a, b) @ (b, c)` → `(a, c)`

the inner sizes must agree

For a batch of 128 images: `x @ W` computes all 128×512 weighted sums of the hidden layer in one operation.

The dot product written as a loop

```
def naive_vector_dot(x, y):
    assert x.shape == y.shape           # same length
    z = 0.0
    for i in range(x.shape[0]):
        z += x[i] * y[i]
    return z

def naive_matrix_dot(x, y):
    assert x.shape[1] == y.shape[0]     # inner sizes agree
    z = np.zeros((x.shape[0], y.shape[1]))
    for i in range(x.shape[0]):
        for j in range(y.shape[1]):
            z[i, j] = naive_vector_dot(x[i, :], y[:, j])
    return z
```

In practice: `np.matmul(x, y)`, or simply `x @ y`.

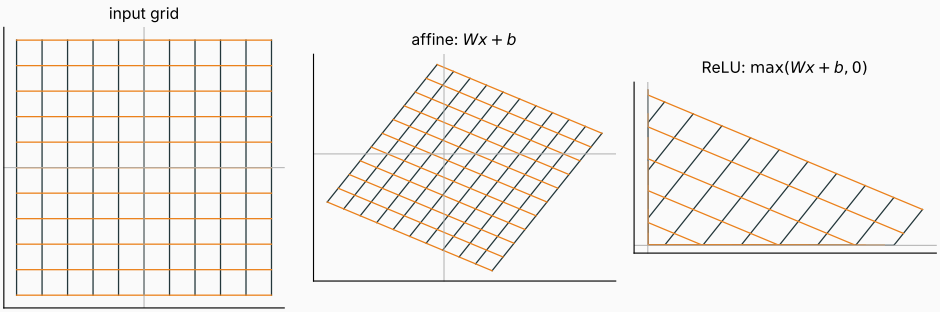
Reshaping and transposing

```
x = np.array([[0., 1.],
              [2., 3.],
              [4., 5.]])      # shape (3, 2)
x.reshape((6, 1))             # one column: 0, 1, 2, 3, 4, 5
x.reshape((2, 3))             # [[0, 1, 2], [3, 4, 5]]
x.T                           # transpose: shape (2, 3), [[0, 2, 4], [1, 3, 5]]

train_images.reshape((60000, 28 * 28))    # flatten every image
```

- `reshape`: the same numbers read row by row, arranged into a new shape; the total count must not change.
- Transpose: rows become columns. This is *not* the same as `reshape((2, 3))`!

Geometric view: what a layer does to the data



- $Wx + b$ moves, rotates, stretches and shears the space: straight lines stay straight (an **affine** map).
- ReLU folds the space along the axes: everything negative is pushed onto 0.
- A deep network = many such stretch-and-fold steps. Chollet's picture: like *uncrumpling a paper ball*, layer by layer, until the classes lie in simple regions.

How a network learns

The training loop

Repeat until the loss is small enough:

1. Take a **batch** of training images `x` with true labels `y_true`.
2. Run the model: `y_pred = model(x)` (**forward pass**)
3. Compute the **loss** — how far `y_pred` is from `y_true`.
4. Find, for every weight, in which direction the loss decreases (**gradient**, backward pass)
5. Change all weights a little in that direction.

Steps 1–3 we already know. Steps 4–5: **gradient descent**.

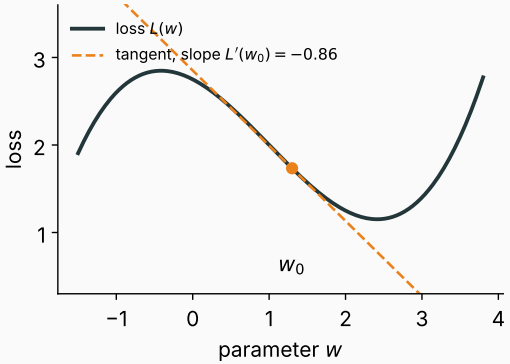
Why not try random changes of weights and keep the good ones? With 407 050 weights that is hopelessly slow. The gradient tells us the good direction for *all* weights at once.

Derivative = slope

- Loss depends on a parameter w: $L(w)$.
- The derivative $L'(w_0)$ is the slope of the tangent line at w_0:

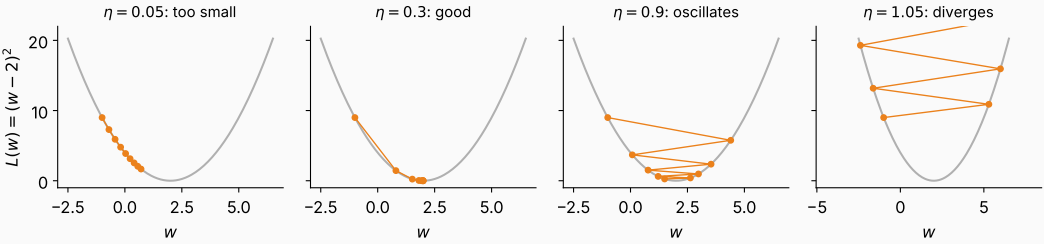
  $$L(w_0 + \varepsilon) \approx L(w_0) + L'(w_0)\,\varepsilon \quad \text{for small } \varepsilon.$$

- $L'(w_0) < 0$: increasing w decreases the loss.
  $L'(w_0) > 0$: decreasing w decreases the loss.
- So: move w against the sign of the derivative.



Gradient descent in one dimension

$$w \leftarrow w - \eta\, L'(w), \qquad \eta > 0 \text{ is the } \textbf{learning rate} \text{ (step size).}$$



Example $L(w) = (w-2)^2$, $L'(w) = 2(w-2)$, start $w = -1$, 8 steps. The learning rate matters a lot: too small — slow; too large — oscillation or divergence.

Your turn: one step by hand

$L(w) = (w-2)^2, \quad L'(w) = 2(w-2), \quad$ start $w = -1, \quad$ learning rate $\eta = 0.3$.

1. Compute $L'(-1)$. Should w increase or decrease?
2. Compute the new w after one step $w \leftarrow w - \eta L'(w)$.
3. Compute the loss before and after the step.
4. Repeat with $\eta = 1.05$. What happens?

Your turn: one step by hand

$L(w) = (w-2)^2, \quad L'(w) = 2(w-2), \quad$ start $w = -1, \quad$ learning rate $\eta = 0.3$.

1. Compute $L'(-1)$. Should w increase or decrease?
2. Compute the new w after one step $w \leftarrow w - \eta L'(w)$.
3. Compute the loss before and after the step.
4. Repeat with $\eta = 1.05$. What happens?

Answers. $L'(-1) = -6 < 0 \Rightarrow$ increase w. New $w = -1 - 0.3 \cdot (-6) = 0.8$. Loss: $9 \to 1.44$.
With $\eta = 1.05$: $w = -1 + 6.3 = 5.3$, loss $9 \to 10.89$ — the step overshoots and the loss grows.

Many parameters: the gradient

With parameters $w_1, w_2, \ldots, w_n$ the loss is $L(w_1, \ldots, w_n)$.

- **Partial derivative** $\dfrac{\partial L}{\partial w_i}$: the slope when only w_i changes and the others are kept fixed.
- **Gradient**: the list of all partial derivatives

$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \ldots, \frac{\partial L}{\partial w_n} \right).$$

  It points in the direction in which L *increases* fastest (locally).

- **Gradient descent**: update every parameter at the same time

$$w_i \leftarrow w_i - \eta \frac{\partial L}{\partial w_i}, \qquad i = 1, \ldots, n.$$

In code: `W -= learning_rate * gradient`, where `gradient` has the same shape as `W`.

Example: linear regression by gradient descent

Data (x_j, y_j), $j = 1, \dots, N$; model $\hat{y} = wx + b$; loss = mean squared error

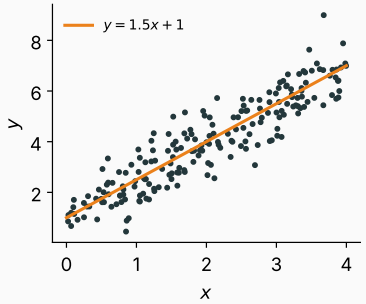
$$L(w, b) = \frac{1}{N} \sum_{j=1}^{N} (wx_j + b - y_j)^2.$$

Partial derivatives (by the chain rule, next lecture):

$$\frac{\partial L}{\partial w} = \frac{2}{N} \sum_j (wx_j + b - y_j)\, x_j, \qquad \frac{\partial L}{\partial b} = \frac{2}{N} \sum_j (wx_j + b - y_j).$$

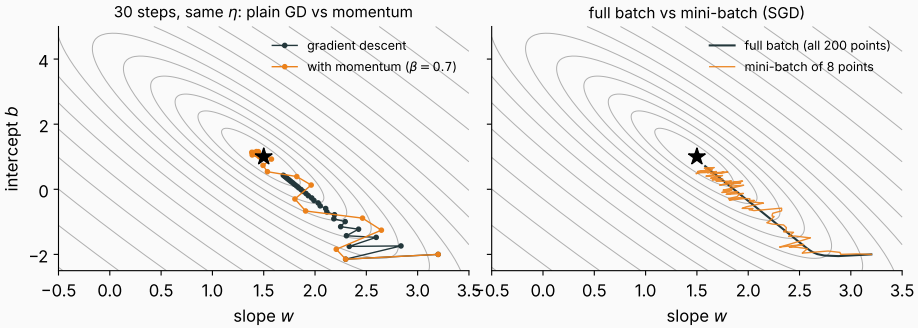
Gradient descent: $w \leftarrow w - \eta\, \partial L/\partial w$, $b \leftarrow b - \eta\, \partial L/\partial b$.

For linear regression the exact solution is known (least squares). For a neural network it is not — but gradient descent works in exactly the same way.



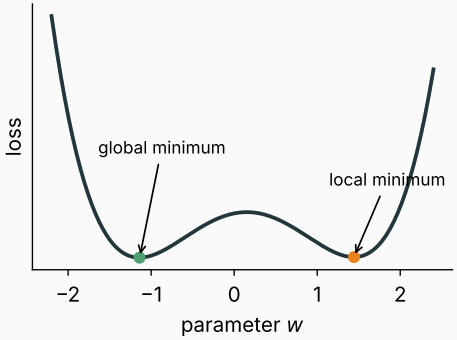
200 simulated points, true line $y = 1.5x + 1$

Stochastic gradient descent and momentum



- **Mini-batch SGD**: compute the gradient on a small random batch (e.g. 128 images) instead of all data. Each step is cheap but noisy (right); this is what `fit(..., batch_size=128)` does.
- **Momentum**: remember the previous step and keep moving in a similar direction, $v \leftarrow \beta v - \eta \nabla L$, $w \leftarrow w + v$ (left: much faster along the narrow valley).

Local minima and optimisers used in practice



- The loss of a neural network is not a nice bowl: gradient descent finds a point where the gradient is (almost) zero, with no guarantee that it is the global minimum.
- The noise of SGD and momentum help to pass small bumps and flat regions. In practice the solutions found for large networks usually work well.
- Popular optimisers (variants of SGD): SGD with momentum, RMSprop, **Adam** (Kingma & Ba 2015). In Keras: `optimizer="adam"`, default learning rate 0.001.

Summary

- A layer is `relu(x @ W + b)`: matrix product, broadcasting, element-wise operation.
- Shape rule for the product: `(a, b) @ (b, c)` $\rightarrow$ `(a, c)`.
- Geometrically, layers stretch and fold the data space.
- Learning = minimising the loss over the weights.
- Gradient descent: $w \leftarrow w - \eta \cdot \partial L / \partial w$ for all weights; the learning rate η is crucial.
- In practice: mini-batches (SGD), momentum, Adam.

Next: how to compute the gradient of a loss with 400 000 parameters — backpropagation.

References

- F. Chollet, M. Watson (2025). *Deep Learning with Python*, 3rd ed. Manning. Sec. 2.3–2.4.
- B. T. Polyak (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics* 4(5), 1–17. (momentum)
- D. P. Kingma, J. Ba (2015). Adam: A method for stochastic optimization. *ICLR 2015*. arXiv:1412.6980.